Biologically inspired artificial intelligence

Project Assumptions

Hyunseok Cho, Jakub Zajac

Silesian University of Technology
Faculty of Automatic Control, Electronics and Computer Science
Informatics

Informatics

June 23, 2026

1 Project Scope

The project assumes that the practical problem is to solve the Gymnasium `MountainCar-v0` environment with a Genetic Algorithm. The agent uses an evolved action table that selects one action for each discretized environment state.

The solution is considered successful when the car reaches the goal position reliably during validation and can be replayed through the graphical environment viewer.

2 Environment Assumptions

Item	Assumption
Environment	`MountainCar-v0` from Gymnasium
Observation values	Car position and velocity
Available actions	`0` = accelerate left, `1` = do nothing, `2` = accelerate right
Goal position	`0.5`
Maximum episode length	`200` steps
Step reward	`-1` for every step until the episode ends

The environment already provides the basic step reward. The implementation stores this value as `total_reward`, so an episode that reaches the goal in 140 steps receives `total_reward = -140`.
An episode that does not reach the goal within 200 steps receives `total_reward = -200`.

3 Data Assumptions

The project does not use an external dataset. All data is generated by running candidate policies inside the Gymnasium environment.

The generated output files are treated as experiment results:

Output type	Purpose
`DATA/results.csv`	Training progress by generation
`DATA/generation_comparison.csv`	Validation metrics for each generation champion
`DATA/randomization_effects.csv`	Robustness check of the saved best policy under fixed random seeds
`DATA/*.png`	Plots and graphical evidence used in the report and presentation
`DATA/generation_champions/*.npy`	Saved best policy from each generation
`DATA/best_individual.npy`	Best policy discovered during the whole training run

4 Policy Representation Assumptions

The continuous MountainCar observation space is discretized before the Genetic Algorithm chooses an action.

Component	Value
Position bins	`20`
Velocity bins	`20`
Total discrete states	`20 * 20 = 400`
Chromosome length	`400` genes
Gene value	One action from $\{0, 1, 2\}$

Each individual is therefore a table of 400 actions.

For a given observation, the code converts the continuous position and velocity into bin indexes, maps those indexes to one state index, and reads the action stored at that gene.

Example:

```
position bin = 8
velocity bin = 11
state index = position_bin * 20 + velocity_bin
state index = 8 * 20 + 11 = 171
chosen action = individual[171]
```

This assumes that a discretized table is expressive enough for the MountainCar task. It also keeps the algorithm simple and easy to explain in the final presentation.

5 Fitness Function Assumptions

The fitness function assumes that a good policy should do three things:

1. Finish the episode in fewer steps.
2. Move farther toward the goal, even if it does not reach the goal yet.
3. Receive an extra reward when it reaches the goal.

The implemented fitness formula is:

```
fitness = total_reward + progress_bonus + goal_bonus
```

The components are calculated as:

```
total_reward = sum of Gymnasium rewards
             = -1 * steps_taken

goal_progress = clamp(
    (max_position - start_position) / (GOAL_POSITION - start_position),
    0,
    1
)

progress_bonus = 500 * goal_progress

goal_bonus = 200 if the goal is reached
goal_bonus = 0 otherwise
```

Example 1 – unsuccessful but improved movement:

```
start_position = -0.50
max_position = 0.00
goal_position = 0.50
steps_taken = 200
reached_goal = False

total_reward = -200
goal_progress = (0.00 - (-0.50)) / (0.50 - (-0.50)) = 0.50
progress_bonus = 500 * 0.50 = 250
goal_bonus = 0
fitness = -200 + 250 + 0 = 50
```

Example 2 – successful policy:

```
start_position = -0.50
max_position = 0.50
goal_position = 0.50
steps_taken = 140
reached_goal = True
```

$$\text{total\_reward} = -140$$
$$\text{goal\_progress} = 1.00$$
$$\text{progress\_bonus} = 500$$
$$\text{goal\_bonus} = 200$$
$$\text{fitness} = -140 + 500 + 200 = 560$$

This assumption gives more importance to how far the car moves toward the goal. Without the progress bonus, many partially useful policies would look almost identical because they would all receive rewards close to `-200`.

6 Genetic Algorithm Assumptions

The Genetic Algorithm assumes that repeated selection, crossover, mutation, and elitism can improve the action table over generations.

Parameter	Value
Population size	80
Number of generations	60
Episodes per individual during training	3
Elite size	6
Tournament size	5
Crossover rate	0.85
Initial mutation rate	0.05
Final mutation rate	0.01

The mutation rate decreases linearly from `0.05` to `0.01`. Early generations therefore explore more aggressively, while later generations preserve stronger solutions with fewer random changes.

Elitism assumes that the strongest individuals in the current generation should survive directly into the next generation. This protects the best discovered policies from being lost through crossover or mutation.

7 Evaluation And Validation Assumptions

Training fitness and validation fitness are treated as different measurements.

Training is used to evolve the population.
Validation is used to compare saved generation champions more fairly.

Evaluation mode	Seeds	Purpose
Training evaluation	Unfixed episode resets	Evolve the population
Generation validation	1042 to 1051	Compare each generation champion under the same conditions
Randomization test	2042 to 2061	Check the saved best policy under additional fixed starts

Because these evaluation modes are different, the best training generation and the best validation champion do not have to be the same.
In the recorded experiment, generation 56 had the highest training fitness, while generation 55 had the best validation fitness.

8 File Organization Assumptions

The repository is organized according to the final submission structure:

Folder	Role
`DOC/`	Reports, assumptions document, and presentation source
`IMPL/`	Python implementation and setup instructions
`DATA/`	Generated plots, CSV files, screenshots, and saved policies